Hangman Team Plan

We're building a Hangman word game in C. Each of us writes one code file and keeps one course role, and everything must be ready for presentations starting Nov 10. Ideally, we will finish all the programming work by Nov 5th to have plenty of time to work on the presentation.

Our program is going to be a multi-file program. Every file will get compiled together which will work because the Idea is when all of the files are compiled together, they are combined into one big source file.

Goal

Our program is a menu-driven Hangman game. It loads words from a text file we'll call words.txt, picks one at random, checks each letter guess, and draws Hangman's body as wrong guesses add up. It scores every game, lets the player play again, and saves results to another text file called scores.txt for a statistics report.

The menu will have six actions:

1. Play
2. Add a word
3. Show the word list
4. Show the score history
5. Show statistics
6. Save

   Option 0 quits.

A player gets 6 wrong guesses, a loss will earn 0 points. Once a player wins, they are scored, the win gives the player 10 points per letter plus 20 points per life left.

End Goals:

- Compiles successfully with "gcc -std=c11 -Wall -Wextra" and shows no warnings
- All six menu actions work, and bad input never crashes it
- All tests pass(8-test minimum), with expected outputs written down
- The README, a 1–2 page report, 5–7 slides, and a demo script are finished
- The GitHub repo shows commits from all five of us
- Each of us can explain every line of our own file

Technical requirements (and where they go):

- Menu with at least six actions: main.c
- Accept and validate guesses: input.c reads one letter, game.c checks it against the word
- Track incorrect attempts: game.c counts the misses (6 max) and draws the hangman
- Select a word and calculate the score: game.c
- Allow replay: main.c, the play-again loop in playGame
- Save and display scores and results: scores.c

- At least six functions, including a helper function: all files
- Arrays and strings: words.c, input.c, game.c
- A struct and an array of structs: GameRecord in hangman.h, used most in scores.c
- Pointers with pass-by-reference: any function that changes main's counts
- Save and load data with files: words.c and scores.c
- Input validation and error messages: input.c, and each file handles its own errors
- A statistics report: scores.c
- At least 8 documented test cases: QA

Grade Rubric:

- Working features + Correctness 35%
- Code quality + Style 20%
- Documentation(README/report) 15%
- Testing, presentation+demo, and participation(questions+feedback) 10% each.

Team Guidelines:

- Use the function names, parameters, and return types in hangman.h exactly
- Change hangman.h only after the group agrees, then tell everyone to pull
- Give functions a one-line comment saying what it does
- If you need a function from someone else's file, try to ask the owner to add it instead of writing your own copy
- Keep each function to one job

Meeting 1 Goals

- Everyone is set up on GitHub, the repo exists, and hangman.h is agreed and pushed (Implementation Lead)
- Every function table is filled in, and all prototypes are in hangman.h (everyone, then Implementation Lead)

Future Meeting Goals

- Every file compiles alone with no warnings and is pushed on its branch (everyone)
- All test cases are written with expected outputs (QA)
- All files are merged, and the full program builds and runs (Implementation Lead)
- All tests pass, and each owner fixes the bugs in their own file (QA runs the tests)
- README, report, slides, and demo script are done (Documentation Lead and Presenter; everyone writes their paragraph)
- Full rehearsal

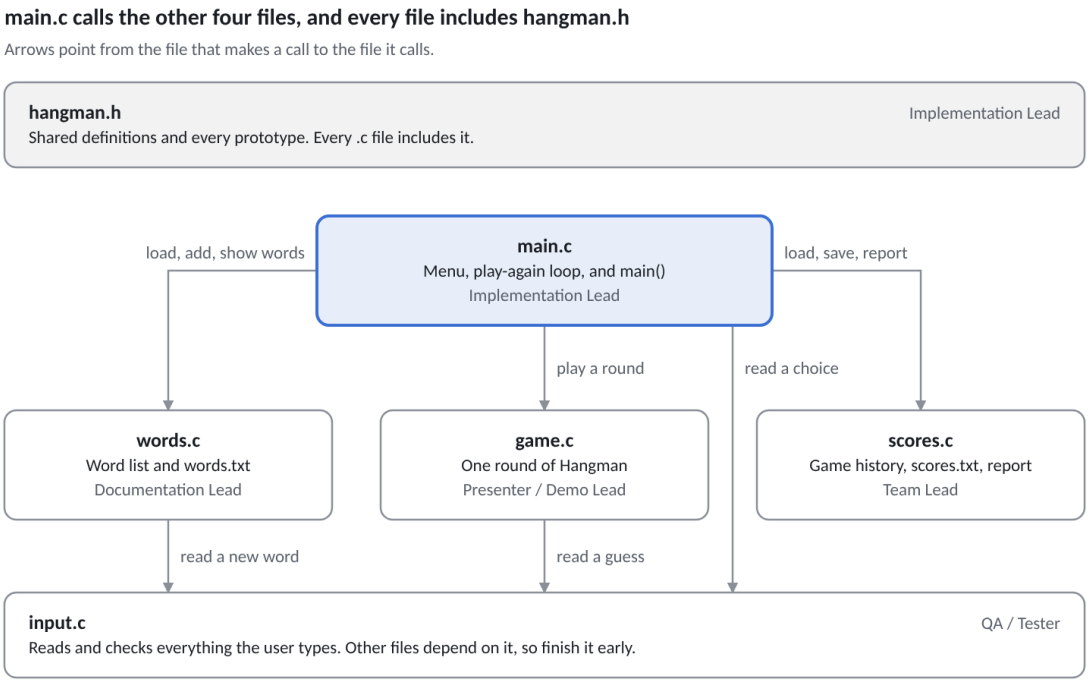
Roles

Each of us writes one code file and keeps one course role through November 10. Everyone codes and everyone presents. Everyone also writes one paragraph about their own file for the report, and answers Q&A questions about it. Your tasks are in your file's section under Code files.

Role	Name	Writes	Responsible for	Presents (10 min total)
Team Lead	Cesar Marquez-Garcia	scores.c	Runs meetings, keeps the timeline and task list, checks that every deliverable is submitted	Problem and users
Implementation Lead	Manuel Barnett	hangman.h, main.c	Creates the GitHub repo, reviews and merges pull requests, keeps the build free of warnings	Design: how the files connect
QA / Tester	Christopher Rodriguez	input.c	Writes every test case, runs all tests after every merge, keeps the bug list	Lessons learned: our biggest bug and its fix
Documentation Lead	Jesse Garcia	words.c	Writes the README (build, run, usage, file formats) and puts the report together	Design: the struct and file formats
Presenter / Demo Lead	Ean Chino	game.c	Makes the 5–7 slides and the demo script, runs the live demo	Demo: 2–3 scenarios, including one error case

Code structure

The program is split into six files, and each file has one owner.



Because every file includes hangman.h, any file can call a function written in another. To build the whole program, compile all five .c files together:

```
gcc -std=c11 -Wall -Wextra main.c input.c words.c game.c scores.c -o hangman
```

Compiling

Check your own file early. The -c flag compiles one file without building the whole program:

```
gcc -std=c11 -Wall -Wextra -c input.c
```

To try your functions before the other files are done, write a small test file with its own main() that calls them, and compile it with your file (add input.c if your file uses it):

```
gcc -std=c11 -Wall -Wextra mytest.c words.c input.c -o mytest
```

Messages to know:

- undefined reference: a function was called, but no file in the command defines it
- multiple definition: two files define the same function
- implicit declaration of function: the prototype is missing from hangman.h, or the file doesn't include it
- conflicting types: the function doesn't match its prototype in hangman.h

Run the program with ./hangman from the folder that has words.txt.

Shared definitions (hangman.h) a.k.a. the header file

Every .c file includes hangman.h, so we agree on it first and only change it as a group. It holds the libraries, the limits, the GameRecord struct, and every prototype from the Code files tables.

Libraries:

- stdio.h: printf, scanf, and the file functions
- stdlib.h: rand and srand, to pick a random word
- string.h: strlen, plus strcpy and strcmp to copy and compare words
- time.h: time(NULL), to seed srand once in main

All four libraries are in our course notes.

Limits (each one is a #define, like #define MAX_WRONG 6):

Name	Value	What it means
MAX_WORDS	50	Most words the list can hold
WORD_SIZE	21	Longest word or name: 20 letters + 1 for the '\0'
MIN_LETTERS	3	Shortest word allowed
MAX_GAMES	100	Most games the history can hold
MAX_WRONG	6	Wrong guesses before a loss
WORDS_FILE	"words.txt"	The word list file
SCORES_FILE	"scores.txt"	The score history file

Use the name in your code, never the number.

One finished game (the GameRecord struct):

Field	What it holds
char player[WORD_SIZE]	The player's name
char word[WORD_SIZE]	The secret word
int won	1 if they won, 0 if they lost
int wrong	Wrong guesses used
int score	Points for this game

In hangman.h, these five fields go inside a typedef struct named GameRecord.

Data main() keeps and passes to the other files:

Variable in main()	What it holds
char words[MAX_WORDS][WORD_SIZE]	The word list, one word per row
int wordCount	How many words are in the list
GameRecord history[MAX_GAMES]	Every finished game
int gameCount	How many games are in history

A function that adds a word or a game gets a pointer to the count (int *wordCount or int *gameCount), so it can change main's variable. That's our pass-by-reference requirement.

File formats:

File	One line holds	Example line
words.txt	One lowercase word of 3–20 letters	capacitor
scores.txt	Player, word, won (1 or 0), wrong guesses, score, separated by spaces	Alex diode 1 2 130

Names and words are always one word of letters only, so spaces can safely separate the fields.

Code files

Each of us writes the functions for our own file. Your table lists the functions your file needs and what each one does. Fill in the blanks with the return type and the parameters, then give it to the Implementation Lead so every prototype goes in hangman.h. If you add a helper function, use the empty row.

hangman.h and main.c (Implementation Lead)

hangman.h holds the shared definitions and every prototype (note: shared definitions are not function definitions). main.c holds the program's data, runs the menu, and connects the other four files.

Code Flow:

- hangman.h: the #include lines everyone needs, the shared definitions, and every prototype from the function tables
- Load words.txt and scores.txt when the program starts
- Show the menu and run the chosen action until the user picks 0
- Play: ask for the player's name, play rounds until they say no, and store each finished game in history
- Ask to save before quitting

Think about:

- Which function from which file does each menu option call?
- What should Play do if the word list is empty?
- Where should srand() go so each run picks different words, and how many times should it run?
- How does a function in another file add a game to history and update gameCount?

Your functions:

Function (fill in the blanks)	What it does	Called by
____ printMenu(____)	Prints the six menu actions and 0 to quit. Takes nothing and returns nothing.	main
____ playGame(____)	Asks for the player's name, plays rounds until they say no, and saves each finished game in history. Takes the word list, the word count, the history, and a pointer to the game count so it can add to it. Returns nothing.	main (option 1)
____ main(____)	Seeds rand once, loads both files, then shows the menu and runs the chosen action until the user picks 0. Returns 0 when the program ends.	the system

input.c (QA / Tester)

input.c reads and checks everything the user types. main.c, words.c, and game.c all depend on it, so it should be one of the first files done.

Code Flow:

- Read a menu choice, and only accept a whole number in range
- Read a guess, and only accept one letter
- Read a yes or no answer
- Read a one-word name or new word, letters only, 20 characters max
- Never crash or get stuck in a loop on bad input

Think about:

- What does scanf return when the user types "abc" instead of a number?
- What's left in the input after scanf reads a number, and how do you clear it?
- Should "D" and "d" count as the same guess?
- What happens if someone types a 30-letter word?
- Which of these checks will words.c and game.c need?

Your functions:

Function (fill in the blanks)	What it does	Called by
____ clearInput(____)	Throws away the rest of the line the user typed, so leftover characters aren't read as the next answer. Takes nothing and returns nothing.	the other input functions
____ readNumber(____)	Shows a prompt and keeps asking until the user types a whole number from min to max. Takes the prompt, min, and max. Returns the number.	main.c
____ readLetter(____)	Shows a prompt and keeps asking until the user types exactly one letter. Takes the prompt. Returns the letter in lowercase.	game.c, askYesNo
____ askYesNo(____)	Keeps asking until the answer is y or n. Takes the prompt. Returns 1 for yes and 0 for no.	main.c
____ readWord(____)	Keeps asking until the user types one word of letters only, 20 characters max, and stores it. Takes the prompt and the char array to store the word in. Returns nothing.	main.c, words.c
____ isAllLetters(____)	Checks a string. Takes the string. Returns 1 if every character is a letter, 0 if not.	readWord, words.c
____ toLowerWord(____)	Changes every letter in a word to lowercase, in place. Takes the word. Returns nothing.	words.c

words.c (Documentation Lead)

words.c keeps the word list and the words.txt file. Whoever owns it also makes the starting words.txt for the demo.

Code Flow:

- Load words.txt at startup, and save it when asked
- Add a new word, and reject duplicates, words under 3 letters, and anything that isn't letters
- Show the word list
- Handle a missing words.txt without crashing
- Make the starting words.txt with at least 10 words

Think about:

- How does main.c find out how many words were loaded?
- How does adding a word update wordCount in main? (This is our pass-by-reference.)
- What should happen when the list is full?
- Should "Diode" and "diode" count as the same word?

Your functions:

Function (fill in the blanks)	What it does	Called by
____ loadWords(____)	Opens words.txt and reads the words into the word list, skipping bad ones. Takes the word list. Returns how many words it loaded, or 0 if the file is missing.	main.c
____ saveWords(____)	Writes every word to words.txt, one per line. Takes the word list and the word count. Returns nothing.	main.c
____ addWord(____)	Asks for a new word and adds it if it's valid and not already in the list. Takes the word list and a pointer to the word count so it can add 1 to it. Returns nothing.	main.c (option 2)
____ showWords(____)	Prints every word, or a message if the list is empty. Takes the word list and the word count. Returns nothing.	main.c (option 3)
____ wordExists(____)	Checks if a word is already in the list. Takes the word list, the word count, and the word. Returns 1 if found, 0 if not.	addWord

game.c (Presenter / Demo Lead)

game.c plays one round of Hangman. It's what the audience watches during the demo.

Code Flow:

- Pick a random word, then show it as blanks with the letters guessed and the lives left
- Draw the hangman, adding one body part for each wrong guess, up to 6
- Fill in every matching letter on a right guess, and don't take a life for a repeated letter
- End the round when the word is solved or on the 6th miss, and score it

Think about:

- How do you know when the word is solved?
- Where will you keep the letters already guessed?
- What does main.c need back from a round to record the game?
- How do you turn rand() into a valid spot in the word list?

Your functions:

Function (fill in the blanks)	What it does	Called by
____ pickRandomWord(____)	Picks a random spot in the word list. Takes the word count. Returns a number from 0 to the word count minus 1.	main.c
____ drawHangman(____)	Draws the gallows with one body part per wrong guess. Takes the wrong count. Returns nothing.	showBoard
____ showBoard(____)	Shows the hangman, the word with blanks, the letters guessed, and lives left. Takes those three things. Returns nothing.	playRound
____ alreadyGuessed(____)	Checks if a letter was already guessed. Takes the guessed letters and the letter. Returns 1 or 0.	playRound
____ revealLetter(____)	Fills in every spot where the letter is in the secret word. Takes both words and the letter. Returns how many spots it filled (0 is a miss).	playRound
____ computeScore(____)	Takes the word length, wrong guesses, and whether they won. Returns 10 per letter plus 20 per life left on a win, 0 on a loss.	main.c
____ playRound(____)	Plays one round until the word is solved or the 6th miss. Takes the secret word. Returns the number of wrong guesses.	main.c

scores.c (Team Lead)

scores.c keeps the history of finished games, the scores.txt file, and our statistics report. It's where we use the array of structs the most.

Code Flow:

- Load scores.txt at startup, and save it when asked
- Show every game in a table with lined-up columns
- Show statistics: games played, wins, losses, win rate, average score, average wrong guesses, and the best game
- Handle a missing scores.txt, which is normal the first time the program runs
- Never divide by zero when no games have been played

Think about:

- How do you read one line of scores.txt into one GameRecord?
- How do you keep the decimals when averaging two ints?
- How can printf widths keep the table columns lined up?
- What should the report say when there are no games yet?

Your functions:

Function (fill in the blanks)	What it does	Called by
____ loadScores(____)	Opens scores.txt and reads each line into one GameRecord in history. Takes the history array. Returns how many games it loaded, or 0 if the file isn't there yet.	main.c
____ saveScores(____)	Writes every game in history to scores.txt, one per line. Takes the history and the game count. Returns nothing.	main.c
____ showScores(____)	Prints every game as a row in a table. Takes the history and the game count. Returns nothing.	main.c (option 4)
____ showStatistics(____)	Prints the report: games, wins, losses, win rate, average score, average wrong guesses, and the best game. Takes the history and the game count. Returns nothing.	main.c (option 5)

Test plan

QA writes and runs every test case. We need at least 8, and each one has its steps and expected output written down before it's run. When a test fails, QA tells the file's owner, and the owner fixes it.

Cover these:

- Each of the six menu actions working normally
- Bad input at the menu, like letters or a number out of range
- Bad guesses, like a digit or two letters at once
- A win and a loss, with the right score
- Guessing the same letter twice
- A missing words.txt
- Saving, quitting, and restarting with the data still there
- Statistics before any game is played

Tip: use a words.txt with only one word for game tests, so you know the answer ahead of time.

#	What it checks	What to type	Expected output	Pass/Fail
1				
2				
3				
4				
5				
6				
7				
8				